

Output

Fruit	Color	Count
-------	-------	-------

0	apple	red	10
1	banana	yellow	5
2	cherry	red	7
3	date	brown	3



Note

You can use `select_dtypes(include="object")` to select only the columns in a DataFrame that contain string data. This makes it easy to apply string operations, such as `str.strip()`, only to columns storing text and not to columns with numbers or other types.

Explain Note



Output

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Paris
2	Alice	25	New York
3	Charlie	35	London
4	Bob	30	Paris

Detecting and Correcting Outliers

Outliers are values in your data that are significantly different from most other observations. They can occur due to measurement errors, data entry mistakes, or genuine variability in the data. Outliers can distort statistical analyses, leading to misleading results, so detecting and addressing them is crucial for ensuring data consistency and correctness.

There are several common techniques for detecting outliers in numerical data. The **Interquartile Range (IQR)** method identifies outliers as values that lie far outside the middle 50% of the data. The IQR is calculated as the difference between the third quartile (**Q3**) and the first quartile (**Q1**). Any data point below **Q1** minus 1.5 times the IQR or above **Q3** plus 1.5 times the IQR is typically considered an outlier.

Another popular technique is the **z-score** method, which measures how many standard deviations a value is from the mean. Values with a z-score greater than 3 or less than -3 are often considered outliers.

	value	is_outlier
0	10	False
1	12	False
2	12	False
3	13	False
4	12	False
5	11	False
6	14	False
7	13	False
8	100	True
9	12	False
10	13	False
11	11	False



Challenge: Standardize Categorical Values

When working with real-world data, you often encounter

categorical values that are meant to represent the same thing but are written in different ways. For example, a survey might record responses such as `Yes`, `yes`, and `YES` in the same column. These inconsistencies can cause problems when you try to analyze or summarize your data, since Python and pandas treat these as distinct values. Standardizing these entries is essential to ensure your data is consistent and your results are accurate.

Output

Response

0	Yes
1	no
2	YES
3	No
4	yes
5	NO
6	n0
7	YeS



Challenge: Convert Data Types

Ensuring that each column in your dataset has the correct data type is fundamental for accurate analysis and computation. When columns meant to represent numbers are stored as strings, calculations and statistical operations may fail or produce incorrect results. This is especially common when importing data from CSV files, spreadsheets, or external sources, where values such as numbers or dates are sometimes interpreted as text. Converting these columns to their appropriate data types ensures that you can perform mathematical operations, aggregations, and visualizations without unexpected errors or misleading outcomes.

Output

```
      item price quantity
0  apple  1.20        10
1  banana 0.80        25
2  orange 1.00        15

item      object
price     object
quantity  object

dtype: object
```


Output	
Original DataFrame:	
	score
0	10
1	12
2	13
3	14
4	15
5	16
6	17
7	18
8	100
9	110

Outliers can significantly impact the quality of your data analysis, especially when they arise from errors or rare events that do not reflect typical patterns. When you want to reduce the influence of extreme values while keeping all your data points, replacing outliers with the **median** of the column is a robust technique. The **median** is resistant to the effect of outliers, so it provides a stable replacement value that maintains the overall distribution of your data. This approach is especially useful when you want to avoid losing data by removing rows, and when the **mean** would be skewed by the very outliers you are trying to address.

Aspect	Median of whole column	Median of non-outliers only (~mask)	
Influence of the outliers	Yes (biased upward)	No	
Statistical robustness	Lower	Higher	
Common practice in industry	Rarely used	Standard recommendation	
Matches the spirit of “replace outliers”	Weaker	Stronger	

- Modify the DataFrame so that every value in the given column is converted to lowercase.
- Return the modified DataFrame.

Style	When to use	This task?
<code>df[col] = df[col].str.lower() + return df</code>	Task explicitly says “modify the DataFrame”	✓ Yes
<code>df_copy = df.copy() \ldots</code>	You want a pure function / best-practice library code	✗ No

Aspect	Your version	Improved version
Speed	Slow (Python-level apply)	Fast (vectorised .str.strip())
Handles string dtype	No	Yes
Handles NaNs	Manual check needed	Automatic
Readability	More lines	Cleaner and more “pandas
Interview preference	Acceptable	Preferred

Task wording	Correct approach	This task?
“Do not modify the original DataFrame — work on a copy”	.copy() + return new DF	✔ Yes
“Modify the DataFrame \ldots”	In-place + return df	✘ No

browser_date	user_id	is_checkout	is_purchase	
2017-12-20	6a7617321513	True	False	
2017-12-20	022d871cdcde	False	False	
...	...	...	...	

Query Results	
question	COUNT(DISTINCT user_id)
1. What are you looking for?	500
2. What's your fit?	475
3. Which shapes do you like?	380
4. Which colors do you like?	361

4. Which colors do you like?	201
5. When was your last eye exam?	270
Database Schema	
home_try_on	
name	type
user_id	TEXT
number_of_pairs	TEXT
address	TEXT
Rows: 750	
purchase	
name	type
user_id	TEXT
product_id	INTEGER
style	TEXT
model_name	TEXT
color	TEXT
price	INTEGER
Rows: 495	
survey	
name	type
question	TEXT
user_id	TEXT
response	TEXT
Rows: 1986	
quiz	
name	type
user_id	TEXT
style	TEXT

